מערכות הפעלה

מערכת קבצים

מערכת קבצים

1. קבצים
2. מדריכים (Directories)
3. יישום מערכת הקבצים
4. דוגמאות

קבצים

Files

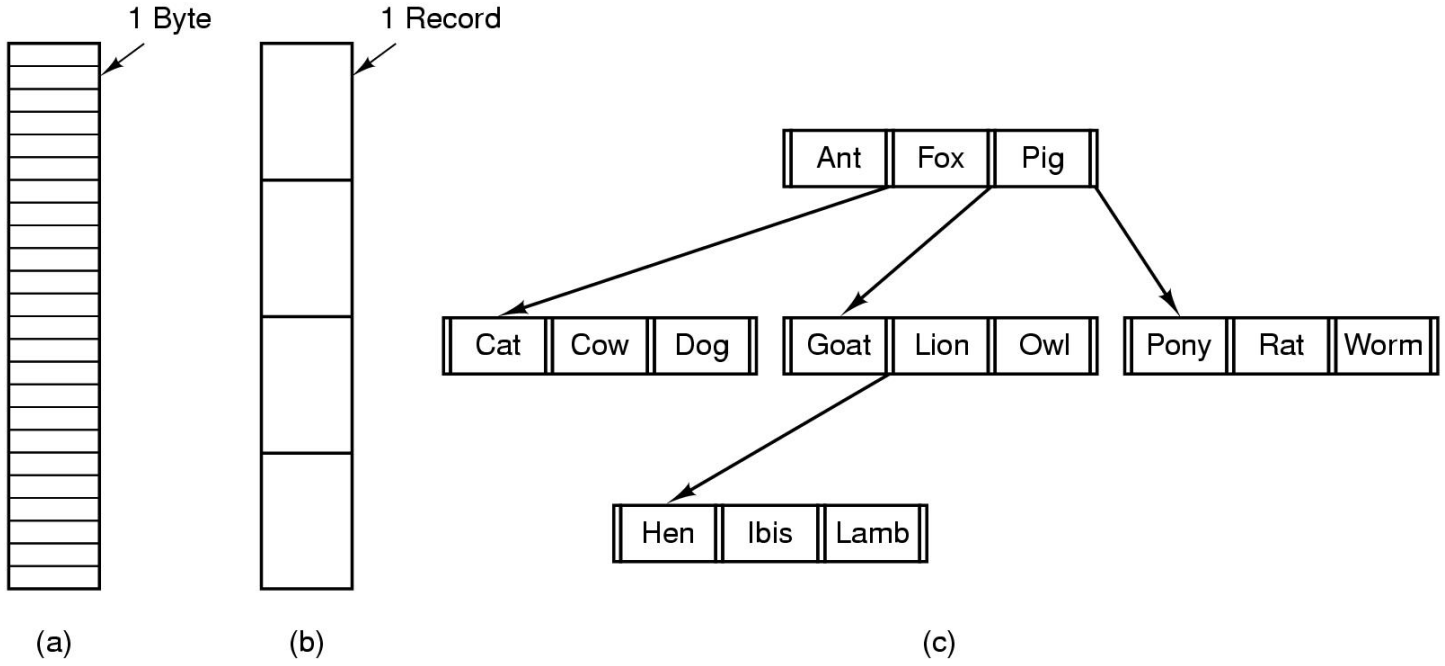
קבצים

1. ארגון ושמירה של כמויות גדולות של נתונים
2. שמירה ואחסון של נתונים בזיכרון המשני
3. שיתוף בין תהליכים

שמות לקבצים

Extension	Meaning
file.bak	Backup file
file.c	C source program
file.gif	Compuserve Graphical Interchange Format image
file.hlp	Help file
file.html	World Wide Web HyperText Markup Language document
file.jpg	Still picture encoded with the JPEG standard
file.mp3	Music encoded in MPEG layer 3 audio format
file.mpg	Movie encoded with the MPEG standard
file.o	Object file (compiler output, not yet linked)
file.pdf	Portable Document Format file
file.ps	PostScript file
file.tex	Input for the TEX formatting program
file.txt	General text file
file.zip	Compressed archive

המבנה הפנימי של קובץ

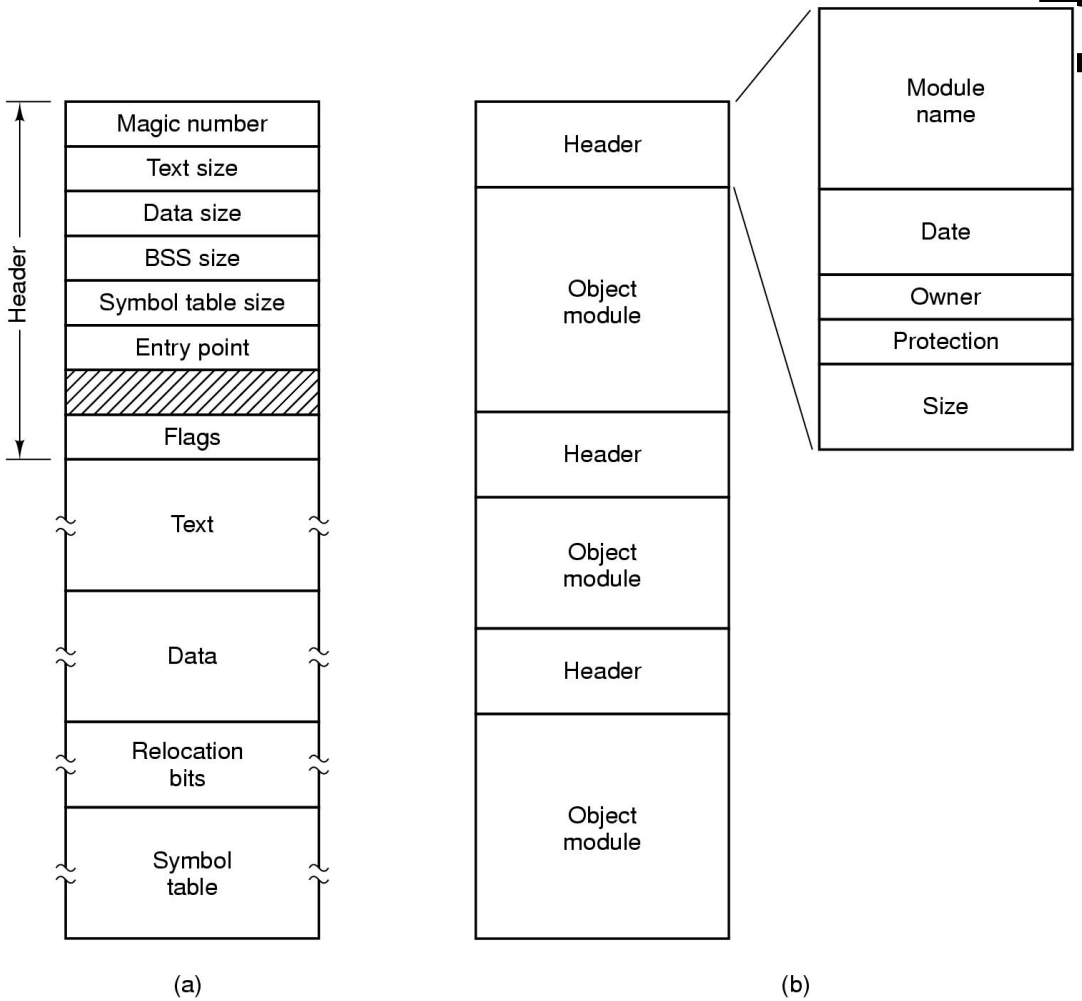


טיפוסי קבצים

- Regular files
  - Ascii
  - Binary

- Special files
  - Block special files
  - Character special files
  - Pseudo-devices

מבני קבצים


Header
Magic number
Text size
Data size
BSS size
Symbol table size
Entry point
Flags
Text
Data
Relocation bits
Symbol table
(a)
Header
Object module
Header
Object module
Header
Object module
(b)
Module name
Date
Owner
Protection
Size

גישה לקבצים

- Sequential access
  - read all bytes/records from the beginning
  - cannot jump around, could rewind or back up
  - convenient when medium was mag tape
- Random access
  - bytes/records read in any order
  - essential for data base systems
  - read can be …
    - move file marker (seek), then read or …
    - read and then move file marker

מאפיינים של קבצים

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Random access flag	0 for sequential access only; 1 for random access
Temporary flag	0 for normal; 1 for delete file on process exit
Lock flags	0 for unlocked; nonzero for locked
Record length	Number of bytes in a record
Key position	Offset of the key within each record
Key length	Number of bytes in the key field
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file has last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

קישור קבצים

- Symbolic link
- Hard link

File Operations

1. Create
2. Delete
3. Open
4. Close
5. Read
6. Write
7. Append
8. Seek
9. Get attributes
10. Set Attributes
11. Rename

12

דוגמא לעבודה עם מערכת הקבצים

```
/* File copy program. Error checking and reporting is minimal. */

#include <sys/types.h>                  /* include necessary header files */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]);       /* ANSI prototype */

#define BUF_SIZE 4096                   /* use a buffer size of 4096 bytes */
#define OUTPUT_MODE 0700                /* protection bits for output file */

int main(int argc, char *argv[])
{
      int in_fd, out_fd, rd_count, wt_count;
      char buffer[BUF_SIZE];

      if (argc != 3) exit(1);           /* syntax error if argc is not 3 */
```

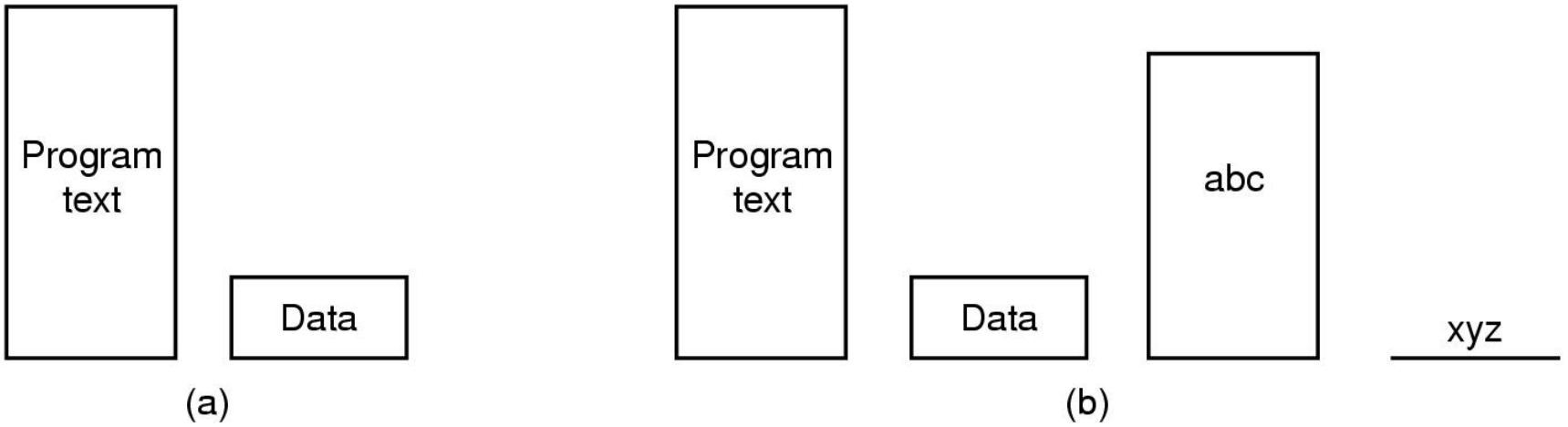
דוגמא לעבודה עם מערכת הקבצים

```
/* Open the input file and create the output file */
in_fd = open(argv[1], O_RDONLY);    /* open the source file */
if (in_fd < 0) exit(2);             /* if it cannot be opened, exit */
out_fd = creat(argv[2], OUTPUT_MODE);  /* create the destination file */
if (out_fd < 0) exit(3);            /* if it cannot be created, exit */

/* Copy loop */
while (TRUE) {
        rd_count = read(in_fd, buffer, BUF_SIZE); /* read a block of data */
if (rd_count <= 0) break;           /* if end of file or error, exit loop */
        wt_count = write(out_fd, buffer, rd_count); /* write data */
        if (wt_count <= 0) exit(4);         /* wt_count <= 0 is an error */
}

/* Close the files */
close(in_fd);
close(out_fd);
if (rd_count == 0)                  /* no error on last read */
        exit(0);
else
        exit(5);                    /* error on last read */
}
```

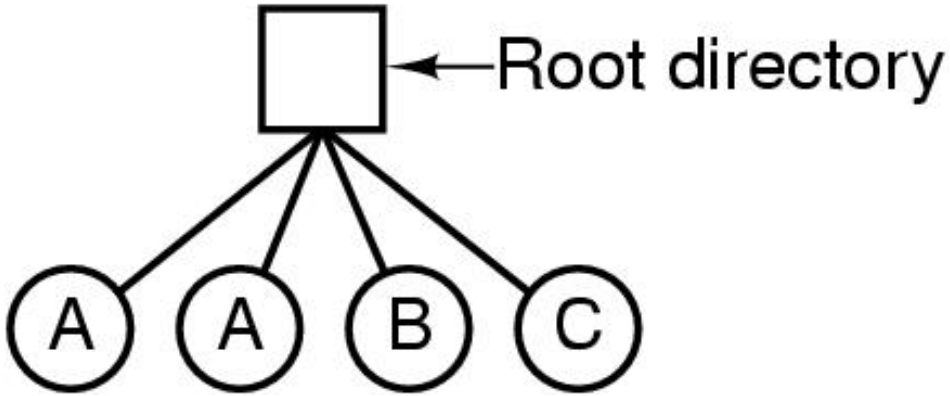
Memory-Mapped Files



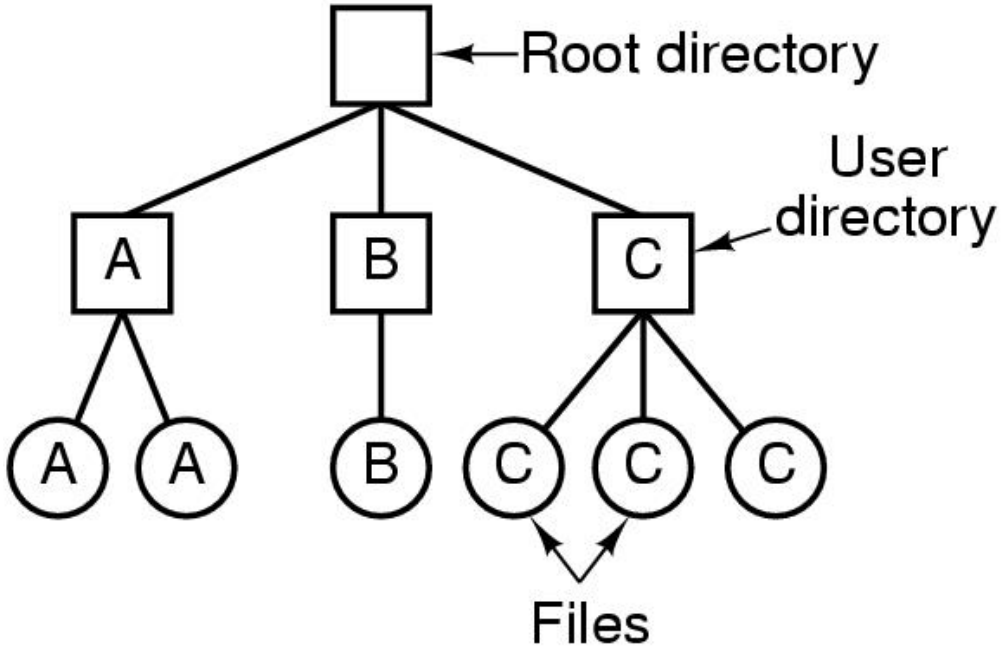
מדריכים

Directories

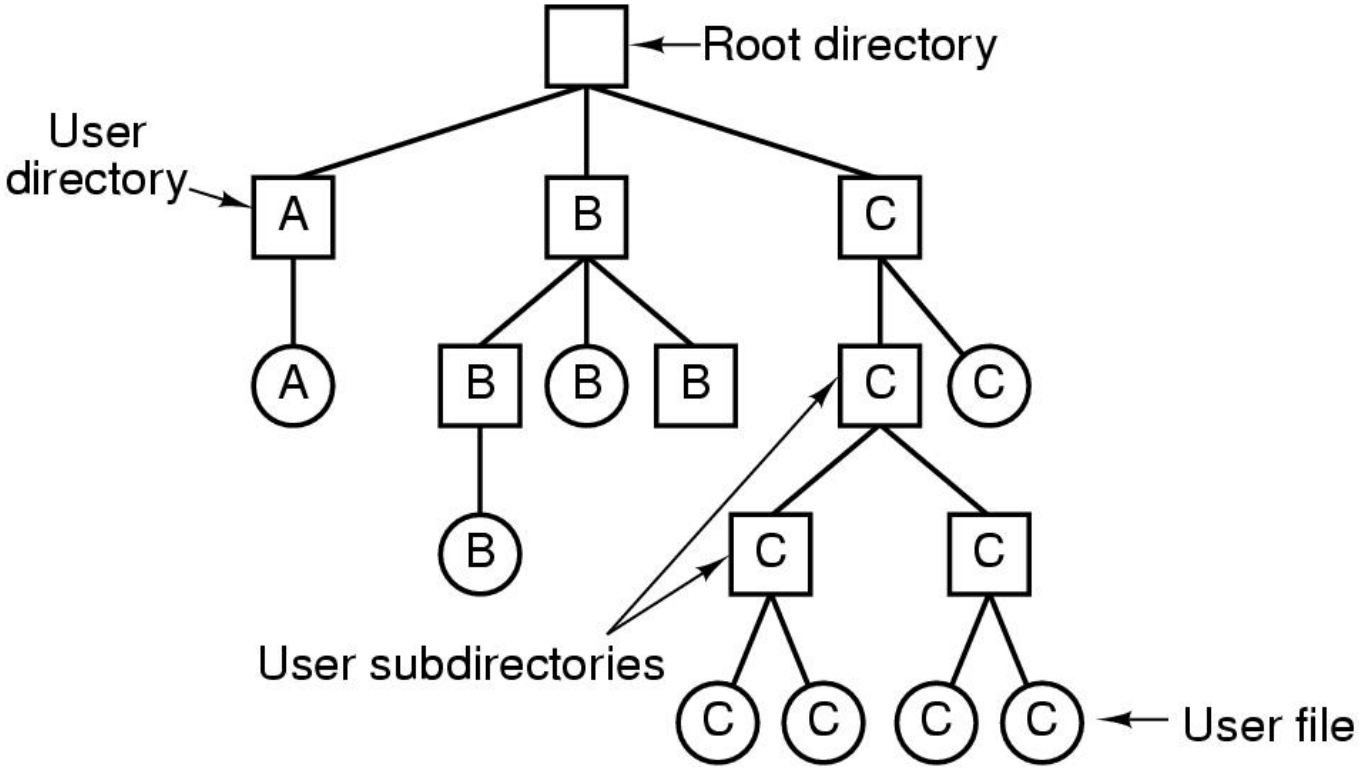
ארגון ברמה אחת



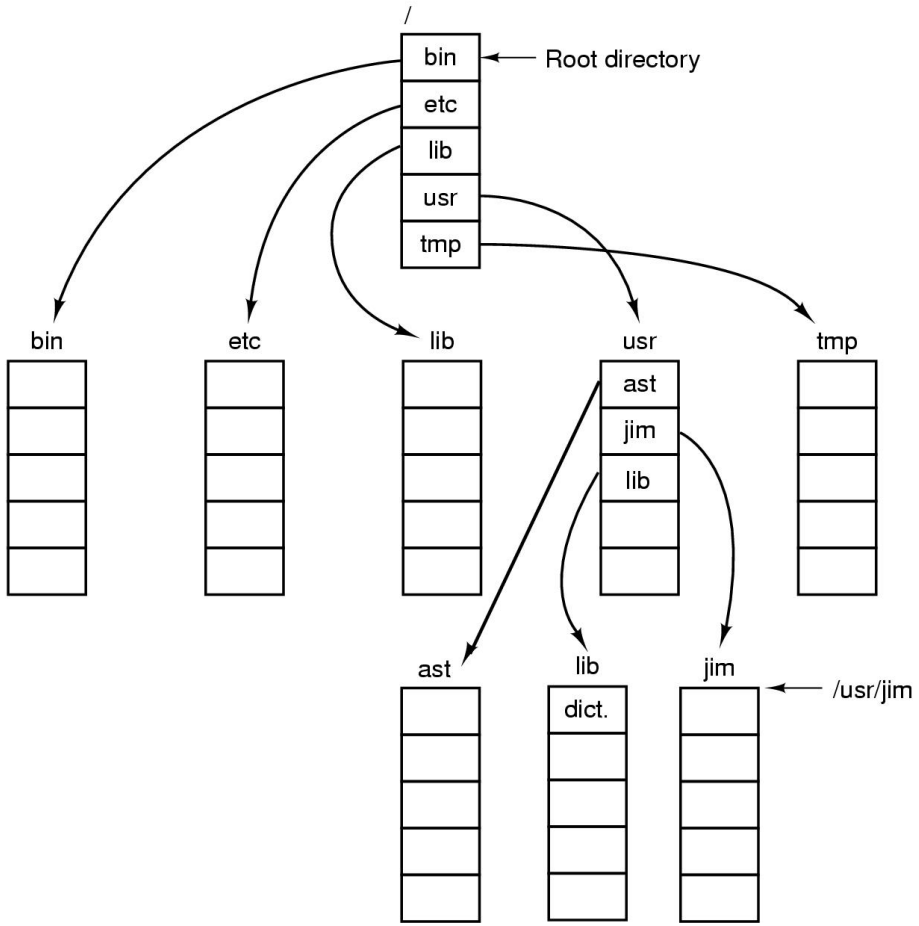
ארגון בשתי רמות



ארגון היררכי



שמות מסלולים



פעולות על מדריכים

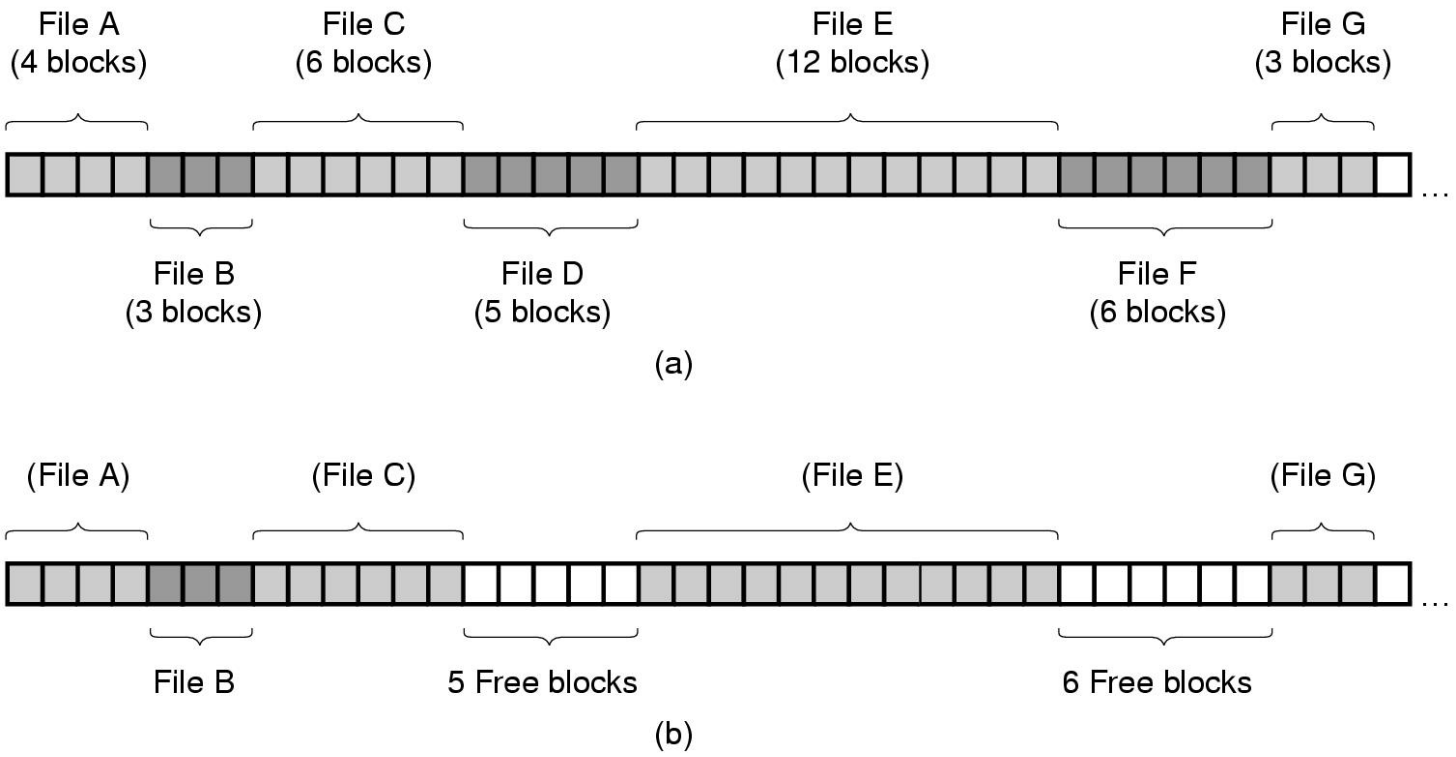
1. Create
2. Delete
3. Opendir
4. Closedir
5. Readdir
6. Rename
7. Link –
   - symbolic link
   - Hard link
8. Unlink

21

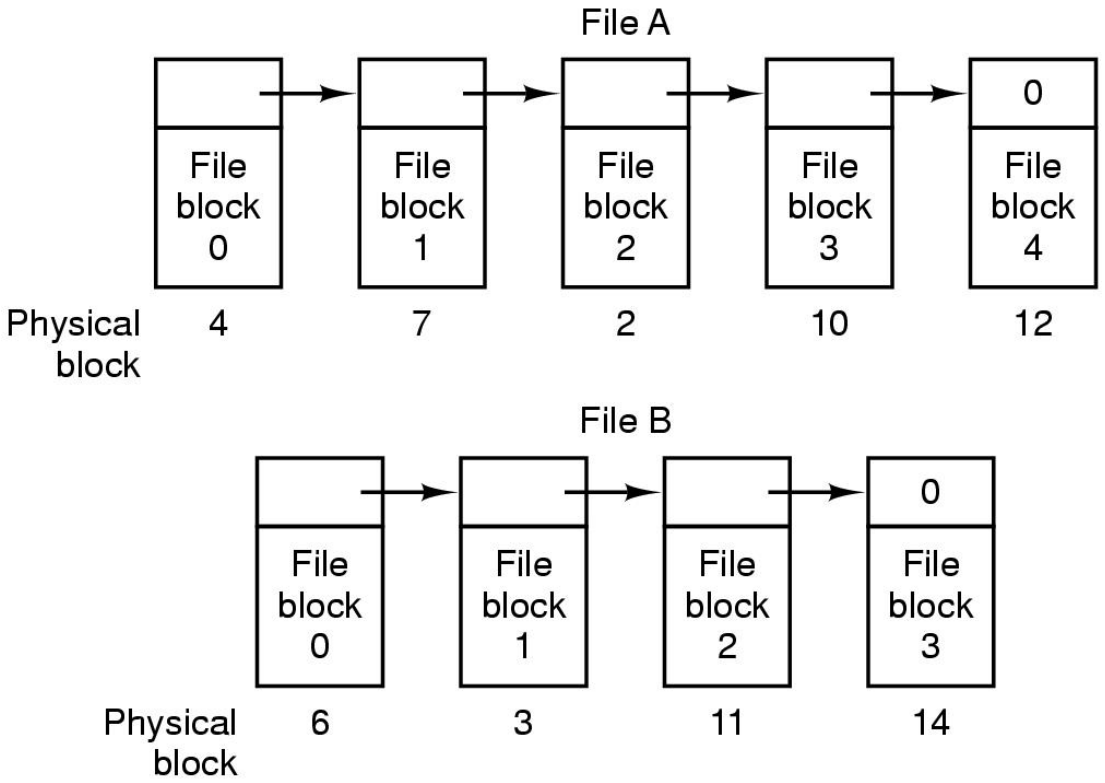
יישום מערכת הקבצים

File system implementation

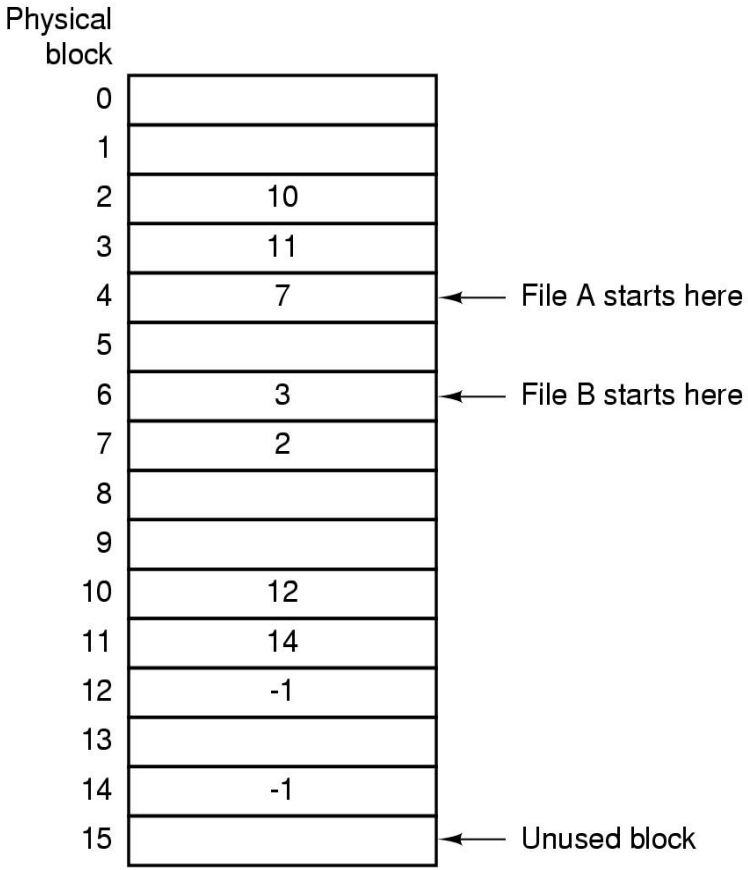
יישום קבצים, הקצאה רצופה



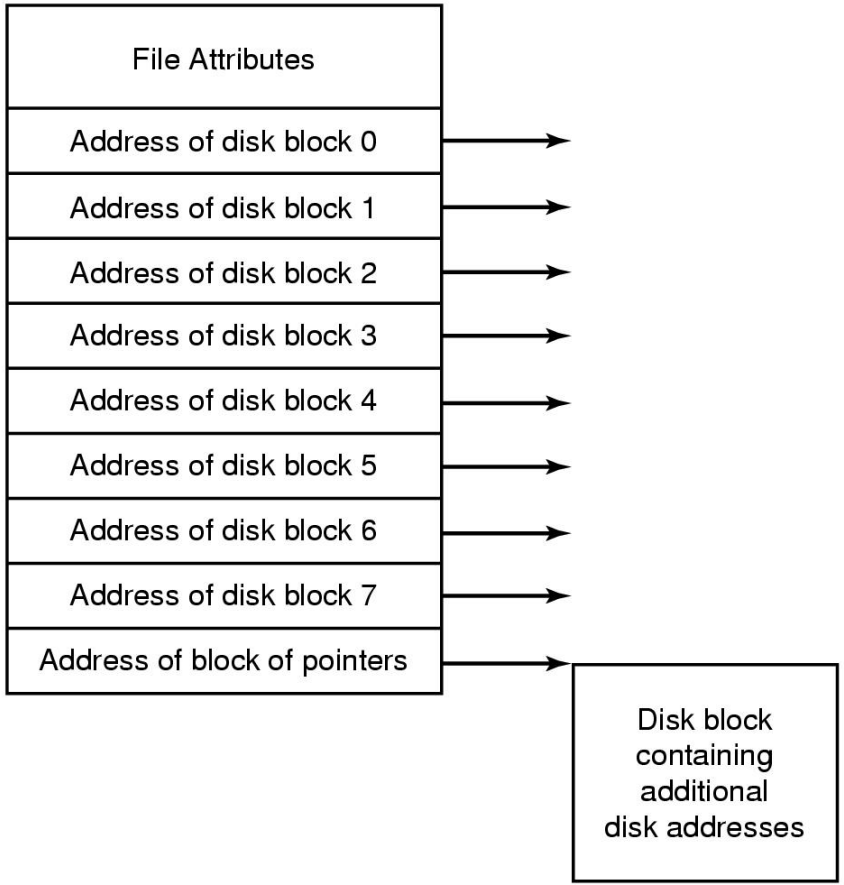
יישום קבצים, רשימה משורשרת



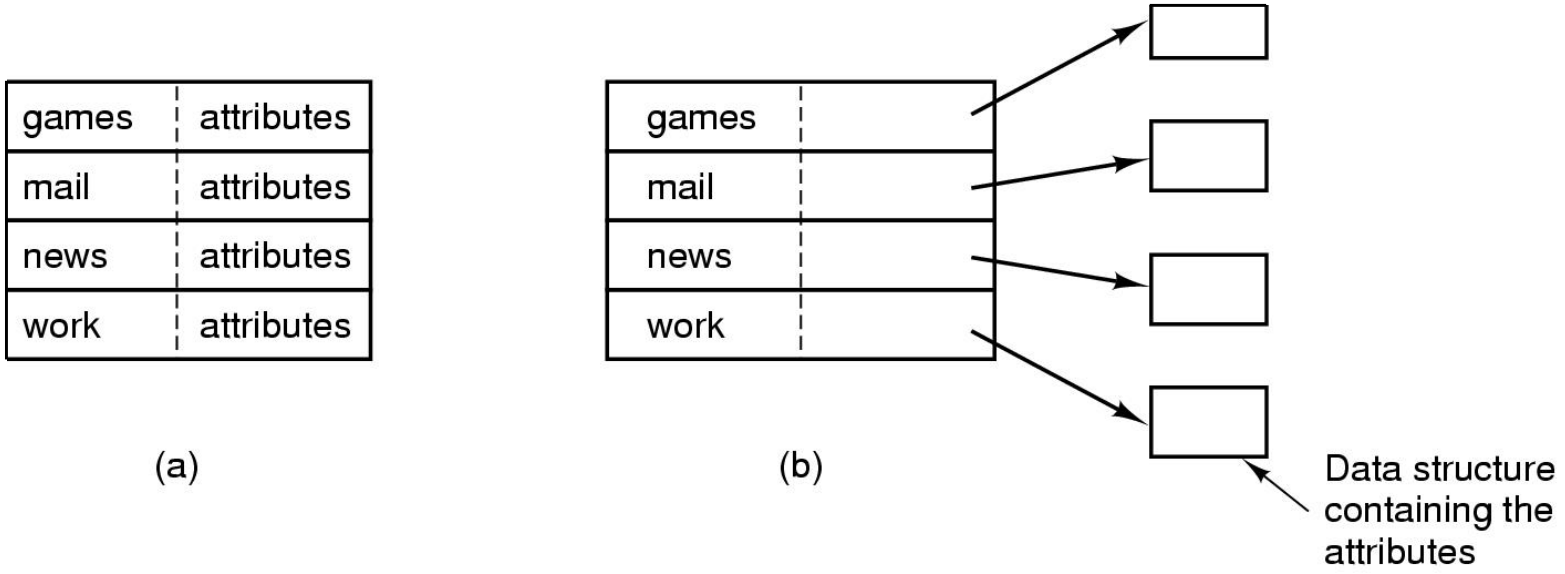
יישום קבצים, רשימה משורשרת וטבלת אינדקס



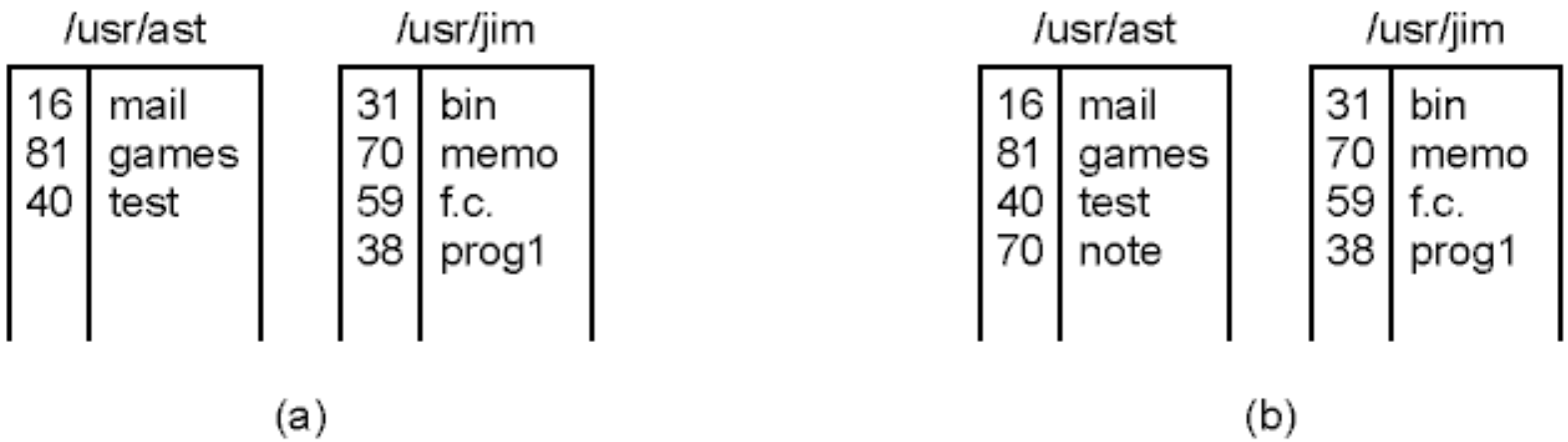
יישום קבצים, שיטת ה- i-nodes



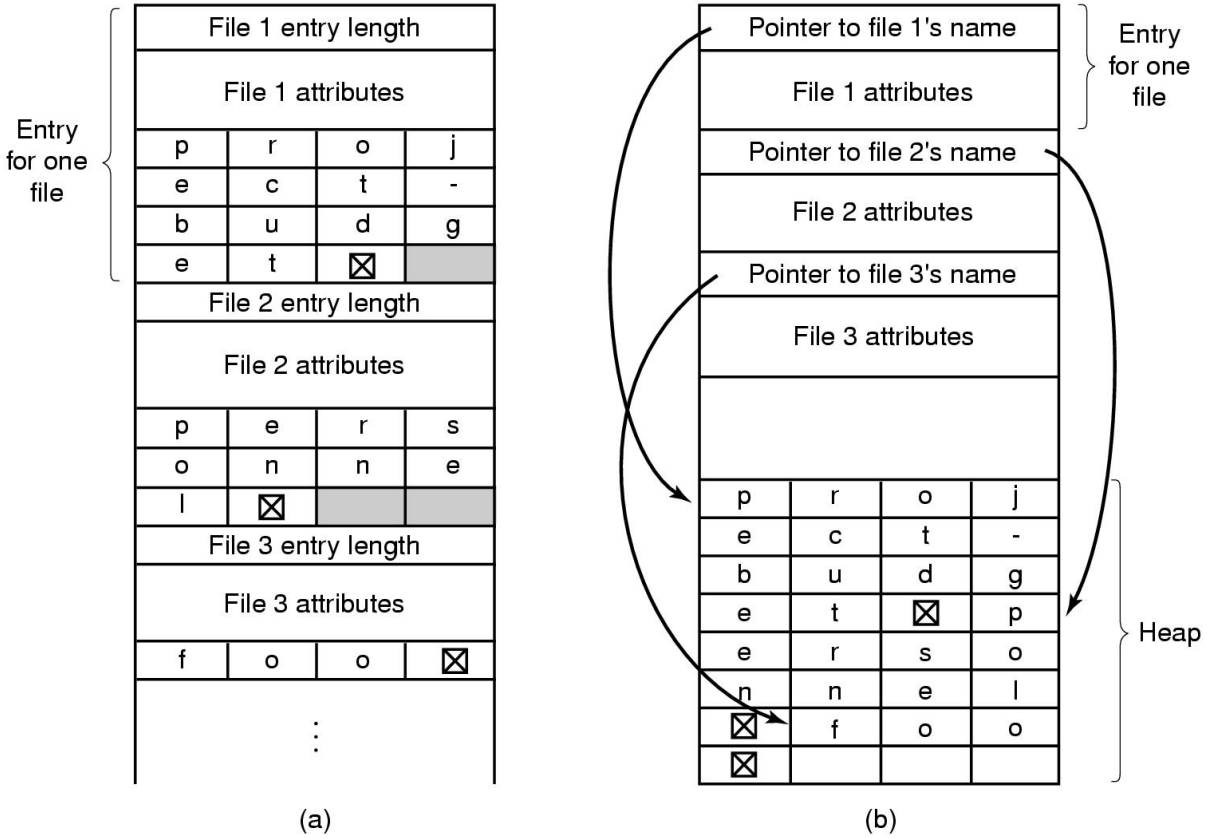
יישום מדריכים



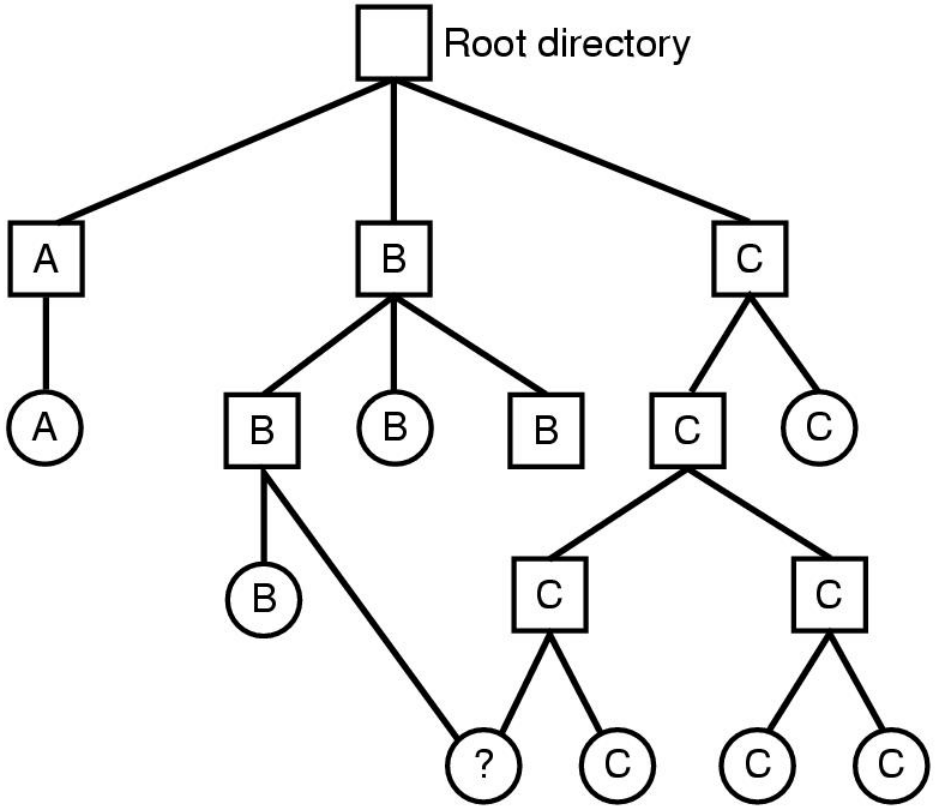
יישום מדריכים



שמות קבצים

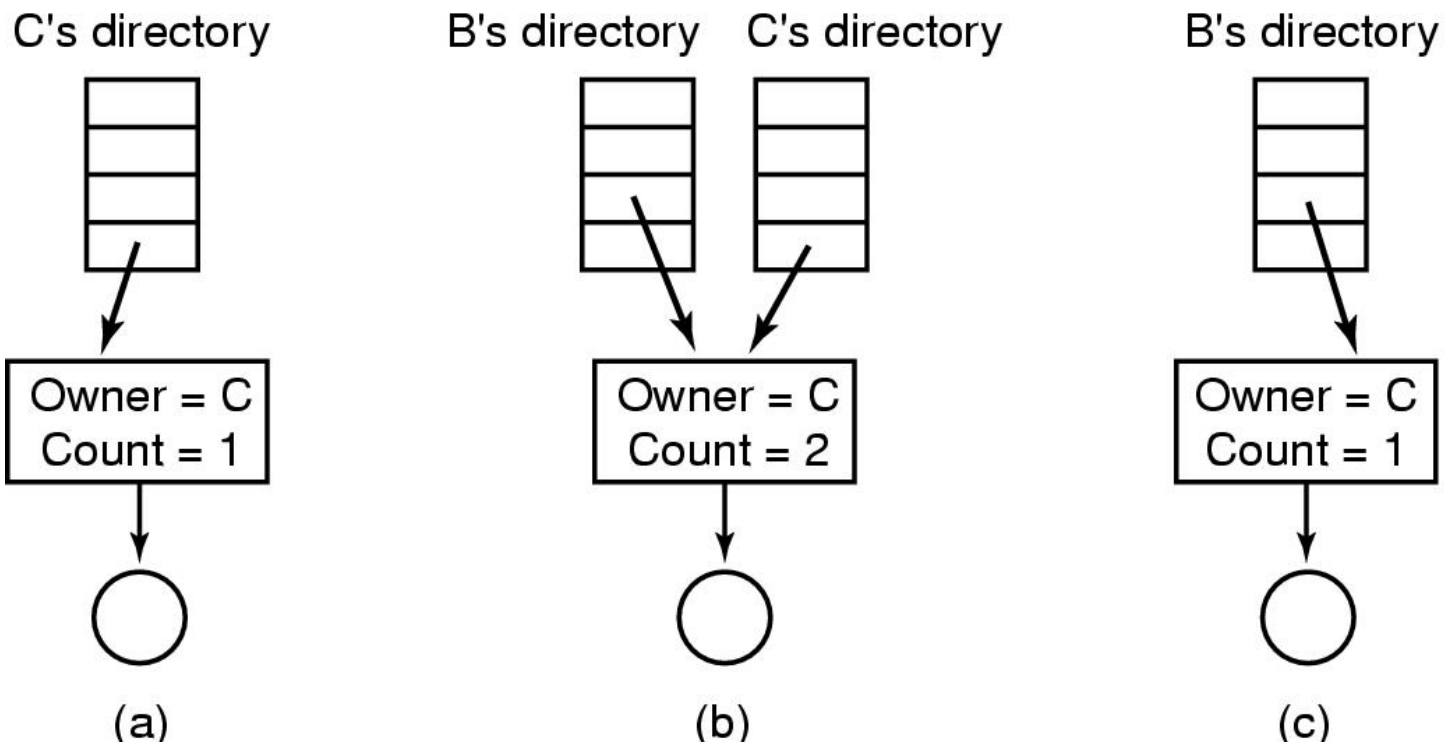

Entry for one file
File 1 entry length
File 1 attributes
p r o j
e c t -
b u d g
e t ⊠
File 2 entry length
File 2 attributes
p e r s
o n n e
l ⊠
File 3 entry length
File 3 attributes
f o o ⊠
(a)
Pointer to file 1's name
File 1 attributes
Pointer to file 2's name
File 2 attributes
Pointer to file 3's name
File 3 attributes
Entry for one file
Heap
(b)

קישור קבצים

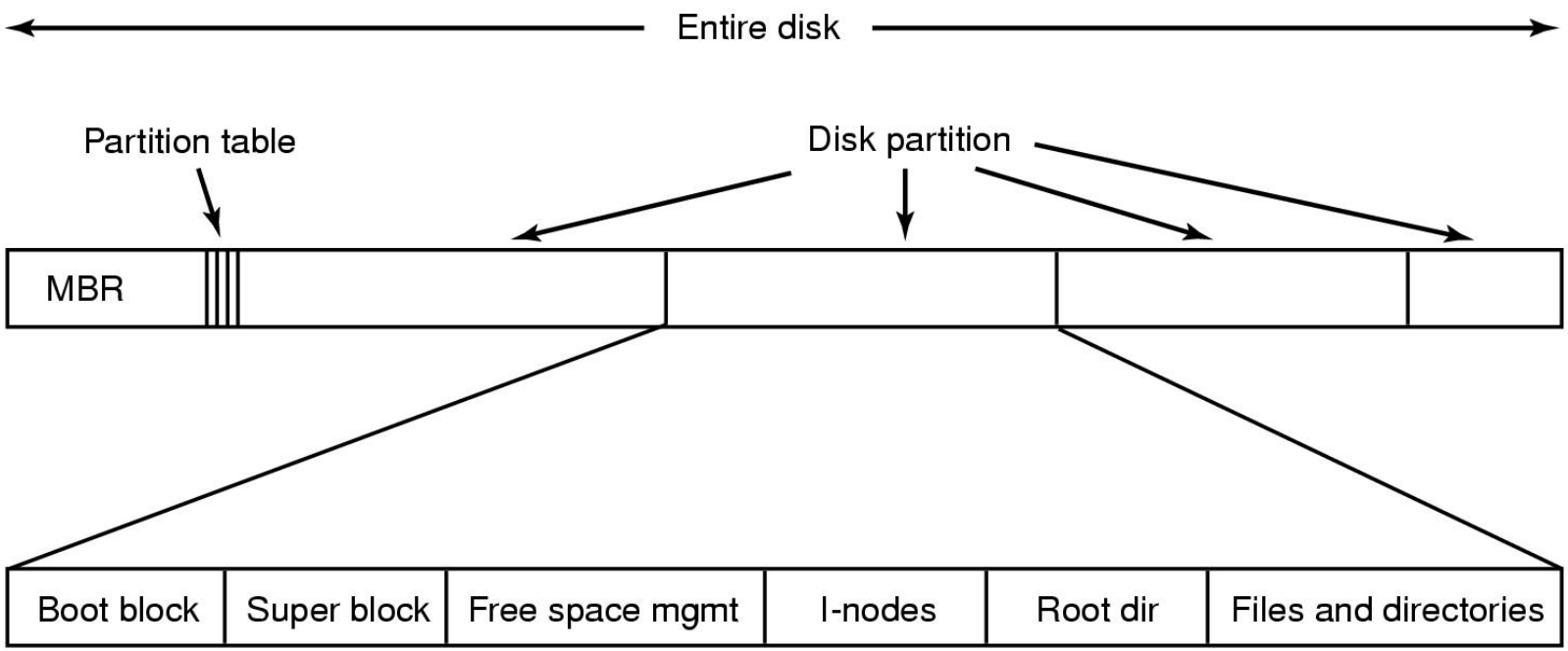


Shared file

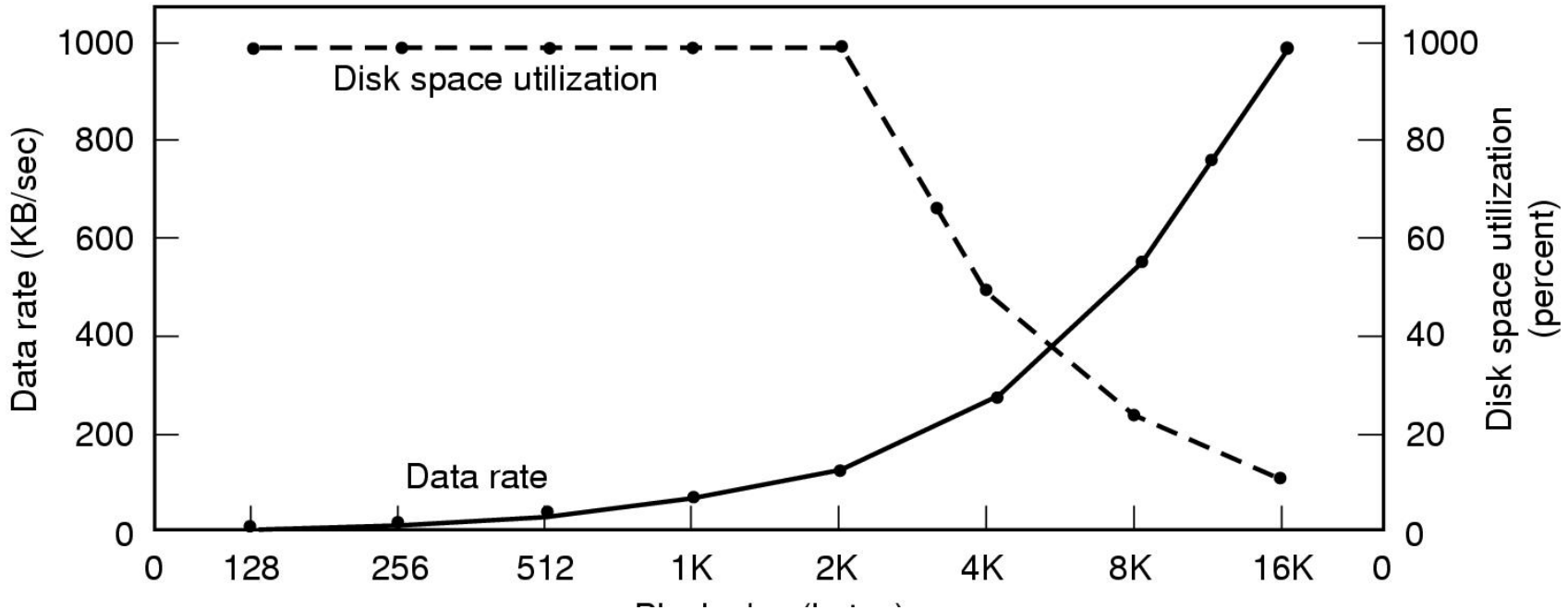
קישור קבצים


C's directory
Owner = C
Count = 1
(a)
B's directory
C's directory
Owner = C
Count = 2
(b)
B's directory
Owner = C
Count = 1
(c)

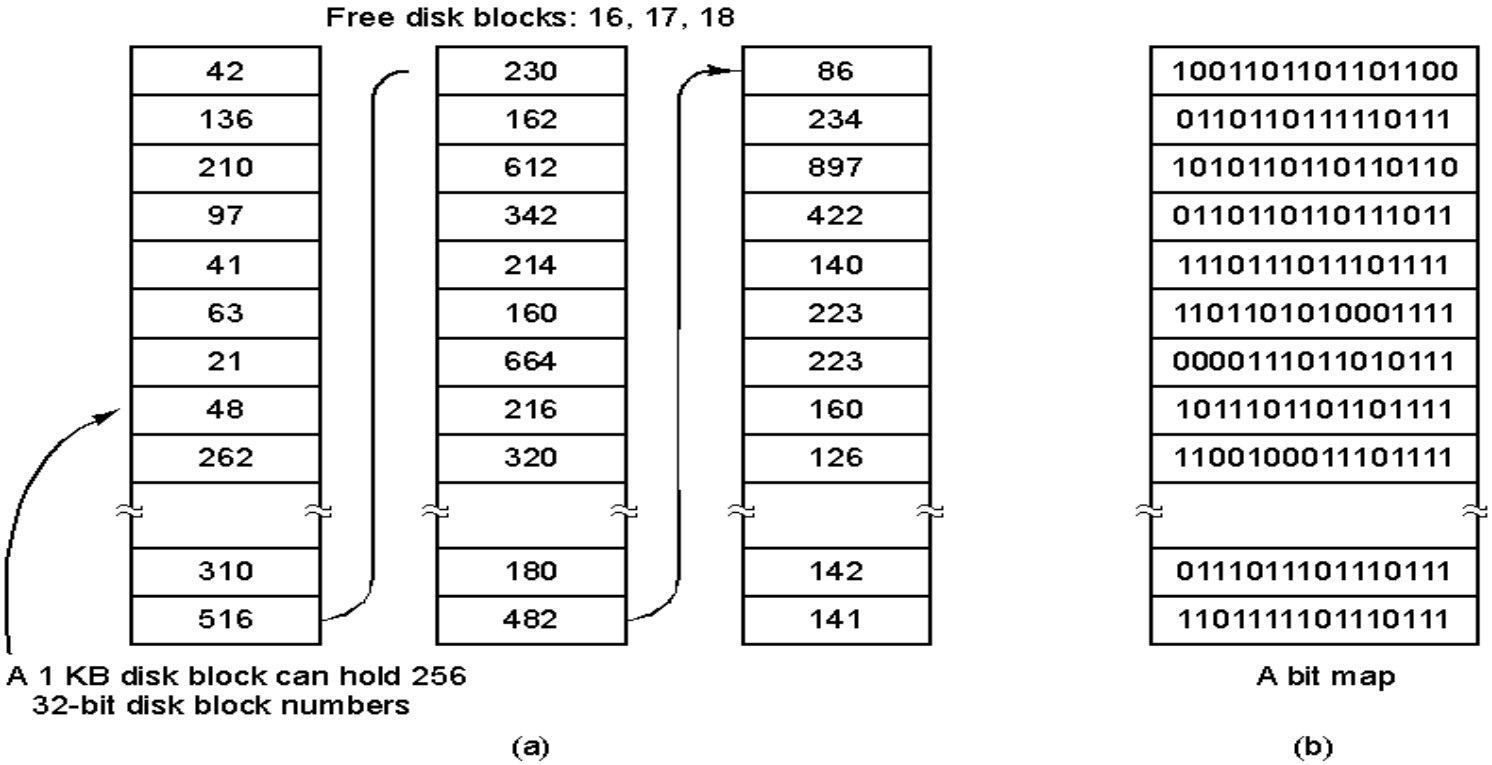
תסדיר מערכת הקבצים



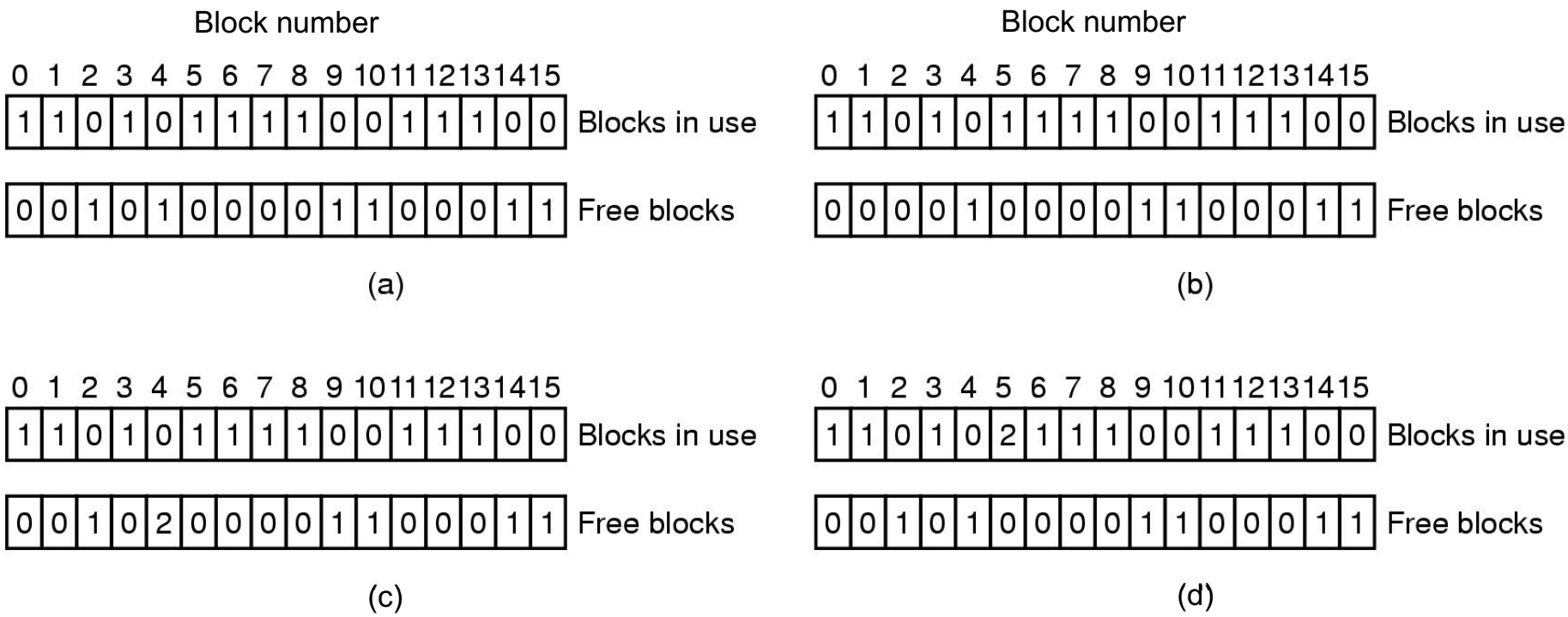
ניהול שטח הדיסק



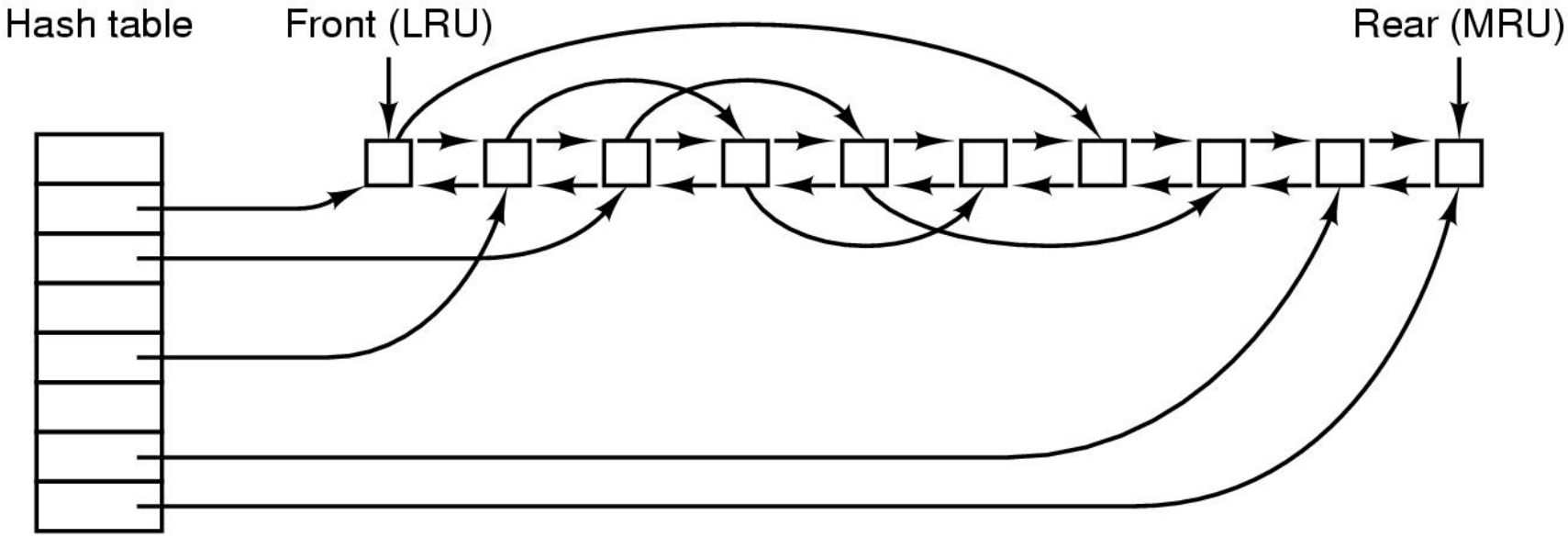
ניהול שטח הדיסק



עקביות מערכת הקבצים (Consistency)



ביצועים של מערכת הקבצים

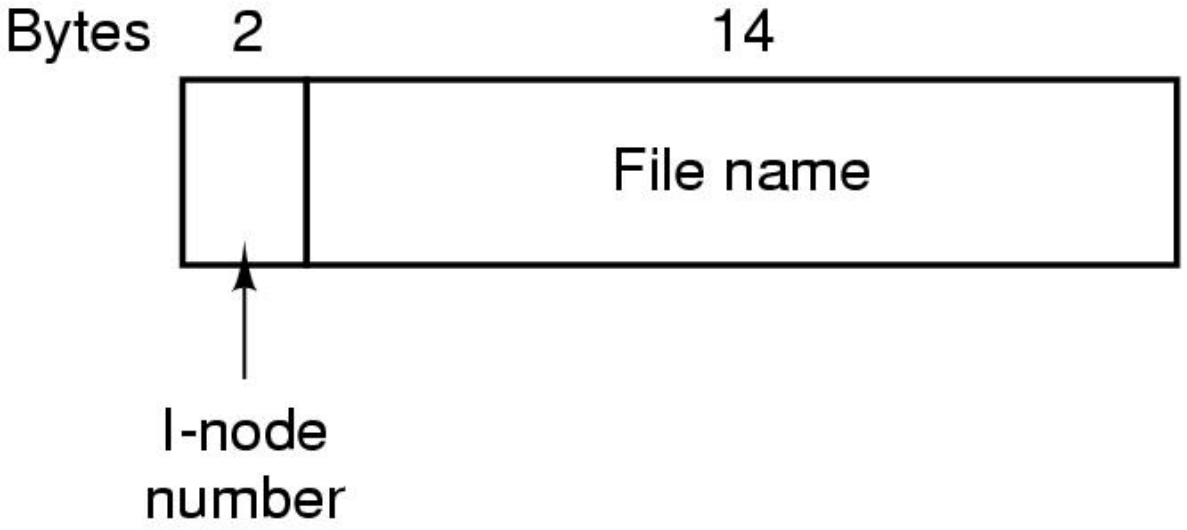


זיכרון מטמון

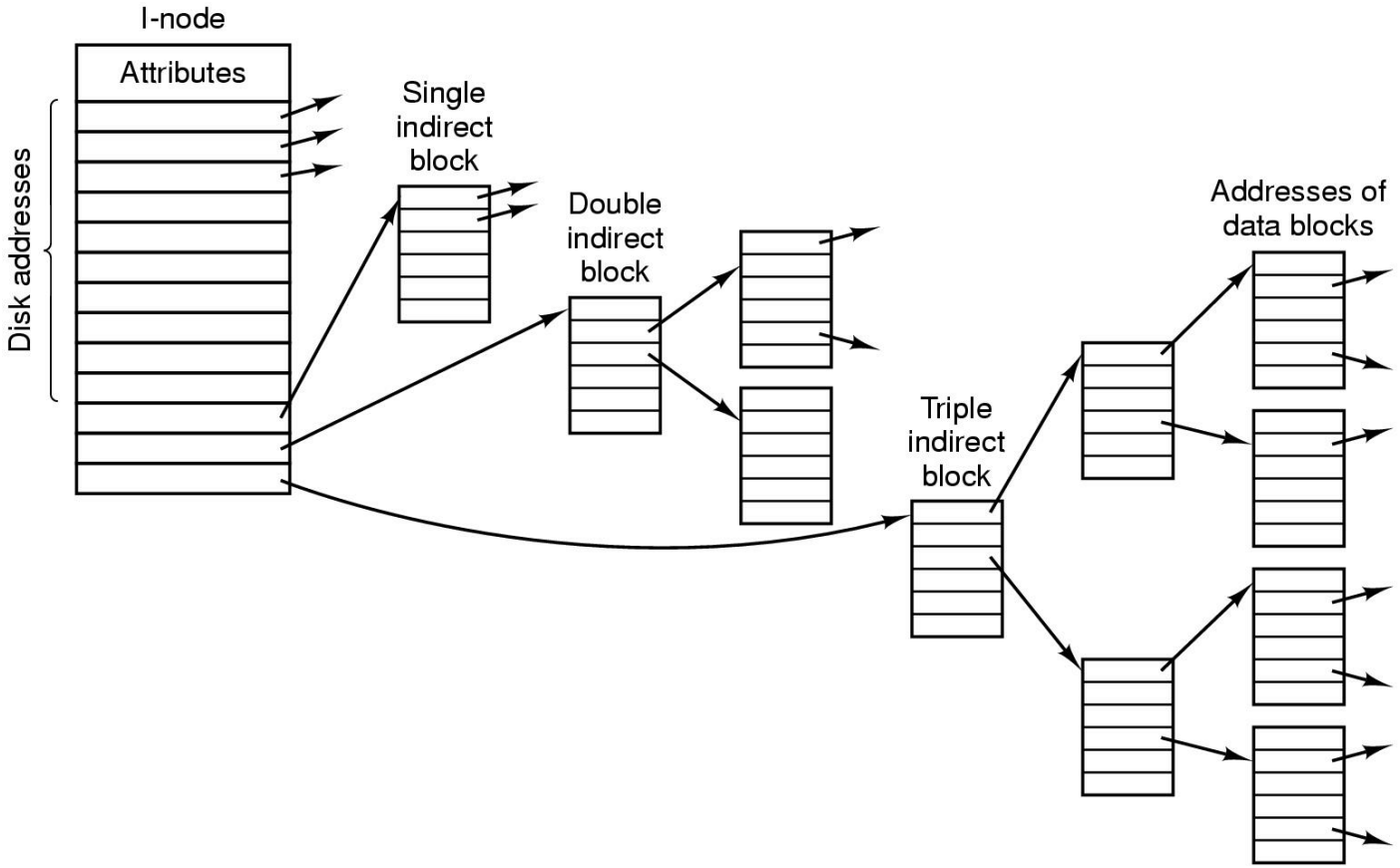
$H * T1 + (1-H) * (T1 + T2) = H*T1 + T1 – H*T1 + (1-H) * T2 =$
$= T1 + (1-H)*T2$

H=hit rate ratio
1-H=miss rate ratio
T1=high level memory access time
T2=low level memory access time

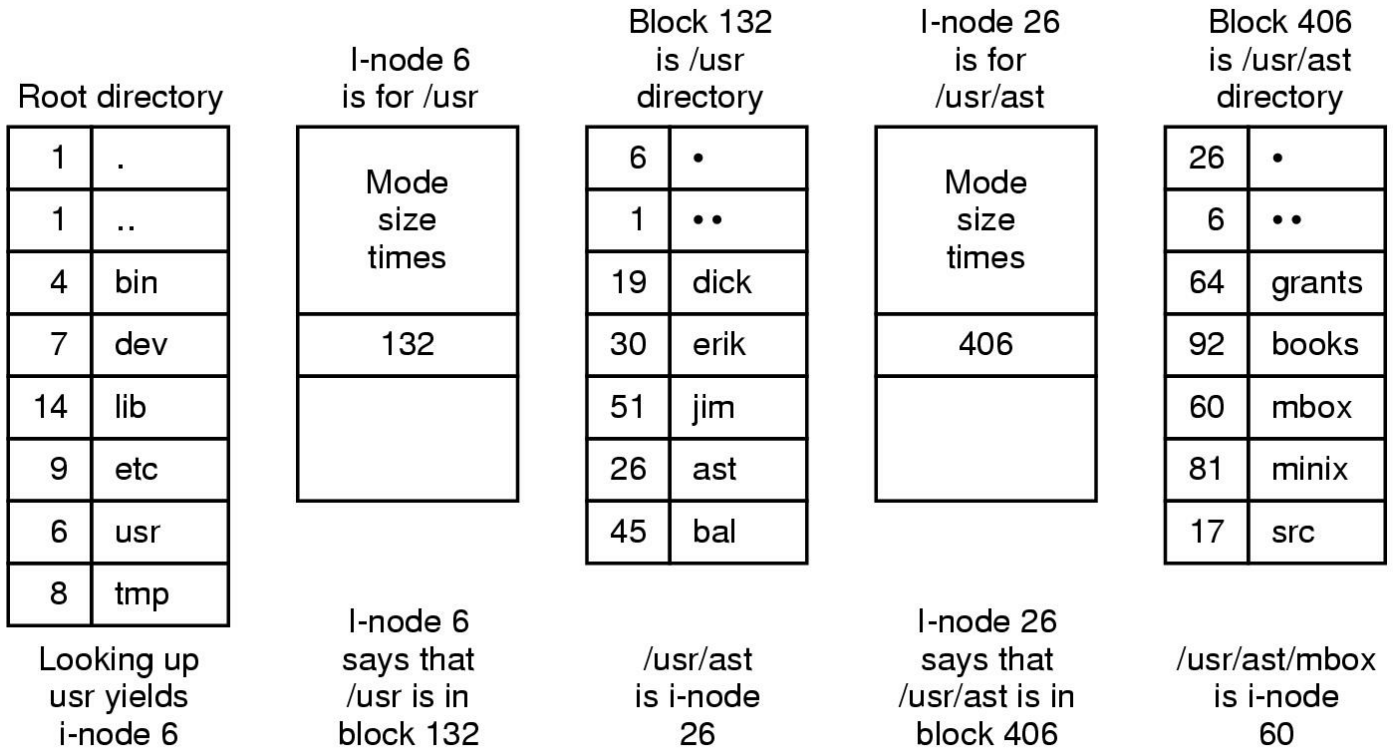
The UNIX V7 File System (1)



The UNIX V7 File System (2)



The UNIX V7 File System (3)



Questions

שאלות

שאלה 1

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה i-node.

- גודל הבלוק במערכת הקבצים הוא 2 kbyte
- כתובת הבלוק היא 8 בתים (bytes)
- יכולים להחזיק ישירות כתובת של בלוק בדיסק i-node שדות של ה 10
- שדה נוסף אחד נועד להחזיק את הכתובת של ה single-indirect block
- עוד שדה נוסף אחד נועד להחזיק את הכתובת של ה double-indirect block
- ועוד שדה נוסף אחד נועד להחזיק את הכתובת של ה triple-indirect block

מהי כמות הבלוקים שדרושה להחזקת קובץ זה .kbytes גודלו של קובץ מסוים במערכת 632
?(של הקובץ i-node לא כולל את הבלוק שמכיל את ה) במערכת הקבצים

א) 316
ב) 317
ג) 318
ד) 319
ה) 320

שאלה 2

מהו מספר פעולות הקריאה של בלוקים מהדיסק שצריך לבצע בהרצת השורה הזאת

```
fd=open(“/a/b/c”);
```

במערכת הפעלה UNIX, כאשר:

- כבר קיים על הדיסק קובץ
- ניתן להניח כי קובץ ספרייה הוא בלוק אחד
- גודלו של כל הבלוקים (למעט ה super block והבלוק המכיל את ה i-node של ספריית השורש) אינם נמצאים בזיכרון מראש.
- קריאות המערכת אינן נכשלות
- נמצאים בבלוקים שונים i-nodes - ה

תשובה

- במערכת ההפעלה UNIX ספריית השורש "/" נמצאת ב- i-node שמספרו 2. לפי ההנחה, הבלוק שמכיל i-node זה כבר נמצא בזיכרון. לכן הבאת הבלוק של ספריית השורש תצריך רק פעולת קריאה אחת.
- בבלוק של ספריית השורש בודקים מהו מספר ה- i-node של הספרייה a ומביאים את הבלוק המכיל אותה, ולאחר מכן – את הבלוק המכיל את קובץ הספרייה a עצמו. מספר הקריאות מהדיסק הוא 2.
- בבלוק של ספרייה a בודקים מהו מספר ה- i-node של הספרייה b ומביאים את הבלוק המכיל אותה, ולאחר מכן – את הבלוק המכיל את קובץ הספרייה b עצמו. מספר הקריאות מהדיסק הוא 2.
- בבלוק של ספרייה b בודקים מהו מספר ה- i-node של הקובץ c ומביאים את הבלוק המכיל אותה (כלומר, את הבלוק שמכיל את ה- i-node של c). מספר הקריאות מהדיסק הוא 1.

שאלה 3

נתון כי: במערכת קבצים נתונה משתמשים בשיטת i-nodes.

- גודל הבלוק הוא 1 kbyte
- גודל המצביע לבלוק הוא 8 בתים (8 bytes)
- מכיל עשרה מצביעים ישירים ושלושה i-node כל
  מצביעים לא ישירים
  (מצביע אחד - triple-indirect ו single-indirect,, double-indirect)
  מכל סוג

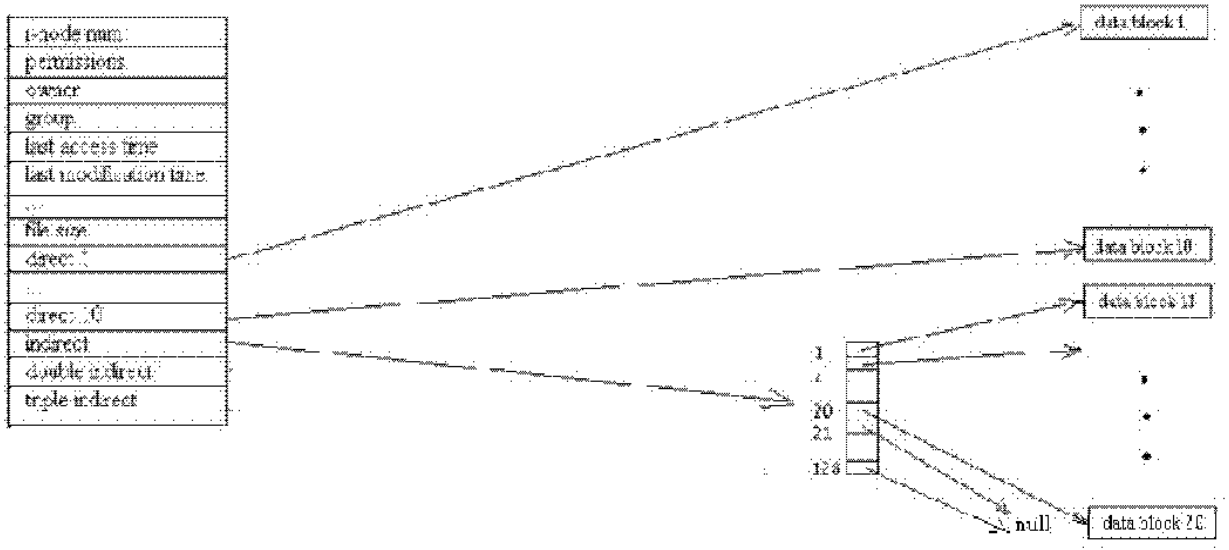
ידוע כי קובץ מסוים משתמש ב- 21 בלוקים (כולל בלוקים של נתונים ובלוקים של מצביעים. אין לוקחים בחשבון את הבלוק המכיל את ה i-node עצמו)

10 מצביעים ישירים מצביעים ישירות על 10 בלוקים של נתונים (מ- 1 עד 10). נותרו לנו עוד 11 בלוקים, שאחד מהם חייב להיות בלוק פנימי של מערכת הקבצים.

בלוק פנימי של מערכת הקבצים יכול להכיל

1kbyte/8Bytes = 2^7

מצביעים לבלוקים אחרים. במקרה שלנו, אלה בלוקים של נתונים מ- 11 עד 20). ראו האיור הבא:



שאלה 4

להלן תוצאות הבדיקה של עקביות מערכת הקבצים

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	Block Number
1	0	0	1	0	0	0	0	1	1	1	1	1	1	0	Blocks in use
0	0	1	0	1	1	1	0	0	1	0	0	0	0	1	Free blocks

מה אפשר להסיק על פי הנתונים הנ"ל

תשובה

- בלוק מס' 2 חסר. כדי להחזיר את מערכת הקבצים למצב עקבי, יש להוסיפו לרשימת הבלוקים הפנויים.
- בלוק מס' 10 הינו בלוק חופשי ומוקצה. כדי להחזיר את מערכת הקבצים למצב עקבי, יש להוציא את הבלוק מרשימת הבלוקים הפנויים.

שאלה 5

נתונות שלוש ספריות שהמידע שלהן נתון בטבלאות הבאות
תקני במערכת זו? שימו לב להבדל בין (path) מהו נתיב
נתיב מוחלט מהשורש לנתיב יחסי
תזכורת: /. מציין נתיב יחסי מהספרייה הנוכחית

directory

26	•
6	••
64	grants
92	books
60	mbox
81	minix
17	src

directory

1	.
1	..
4	bin
7	dev
14	lib
9	etc
6	usr
8	tmp

directory

6	•
1	••
19	dick
30	erik
51	jim
26	ast
45	bal

```
/usr/ast/src
./usr/ast/src
/grants/bin/dev
./grants/bin/dev
/src/ast/usr
./src/ast/usr
```

שאלה 6

מסוג write-trough (buffer cache) האם השימוש בזיכרון מטמון מגדיל את חסינותה של מערכת הקבצים בהשוואה לשימוש בזיכרון מטמון מסוג non-write-trough?

א) כן. וזאת עקב ביצוע מהיר יותר של כתיבות בלוקים לדיסק
ב) לא. וזאת עקב העיכוב בעדכון בלוקים בדיסק
ג) לא. כוון שחסינותה של מערכת הקבצים איננה מושפעת כלל וכלל מזיכרון מטמון
ד) אף תשובה קודמת איננה נכונה

סיכום

סיכום



קבצים
- שמות ומאפיינים
- מבנה
- קישור קבצים



מדריכים (Directories)
- ארגון היררכי
- שמות מסלולים



יישום מערכת הקבצים
- i-nodes
- תסדיר מערכת הקבצים
- ניהול שטח הדיסק
- עקביות מערכת הקבצים (Consistency)



תודה